

Day 2

One Stack, Two Maps

Practical Cyber Security: From First Principles -- Networking Block

Where we are now

Completed already:

- DHCP = how the host got its IP, gateway, DNS server
- DNS = how the host turned a name into an IP

Today:

- Stop teaching OSI and TCP/IP separately
- Start using one unified layer model
- Add IP as the address every layer depends on
- Focus on Application + IP context with live traffic

Why are we learning this?

Students usually memorize two diagrams and still cannot explain one packet.

- OSI gives conceptual detail
- TCP/IP matches real deployments
- Security work needs both at the same time

Remember One user action creates traffic across multiple layers. Today we start reading both models as one stack.

The Unified Map

OSI	TCP/IP
7 Application	Application
6 Presentation	
5 Session	
4 Transport	Transport
3 Network	Internet
2 Data Link	Network Access
1 Physical	

TCP/IP compresses layers. OSI separates responsibilities.

Encapsulation: The big idea

Browser data

- > TCP segment
- > IP packet
- > Ethernet/Wi-Fi frame
- > Signals on the wire

On the receiving host, the process runs in reverse.

- Application cares about content
- Transport cares about conversation
- Internet cares about destination
- Network access cares about local delivery

IP: The number behind the name

You already learned:

- DHCP gives your machine an IP address
- DNS turns a name into a server IP address

Now connect both ideas:

```
Your IP:          192.168.1.105
Destination IP: 93.184.216.34

Browser wants:  https://example.com
Network needs:  93.184.216.34
```

DNS gives the destination number. IP is the addressing system that makes delivery possible.

Where IP sits in both models

```
Application: HTTP, HTTPS, DNS, DHCP
Transport:   TCP or UDP
Internet:    IP
Link:        Ethernet / Wi-Fi / ARP
```

In model language:

- OSI Layer 3 = Network layer
- TCP/IP Internet layer = IP layer

Simple rule:

```
Name  -> DNS resolves it
IP     -> network delivers to it
Port   -> transport selects the service
```

Layers 7, 6, 5: combined view

Today we teach these as one block:

- OSI Layer 7 Application = user-facing network services
- OSI Layer 6 Presentation = format, encoding, compression, encryption
- OSI Layer 5 Session = login state, session setup, continuity
- TCP/IP Application = combines all three

This is why TCP/IP looks simpler, but OSI explains more detail.

Layer 7: Application protocols

Application protocols define the service being used.

Protocol	Purpose	Port
HTTP	web traffic	80
HTTPS	encrypted web traffic	443
FTP	file transfer	21
SSH	secure remote shell	22
RDP	remote desktop	3389

These are user-facing or admin-facing services.

More Layer 7 protocols

Protocol	Purpose	Port
SMTP	sending email	25 / 587
SNTP/NTP	time sync	123
DNS	name to IP	53
DHCP	host configuration	67 / 68

Same layer, different services. TCP/IP groups all of them under Application.

Layer 7: what to inspect

In Wireshark, Layer 7 evidence usually appears as:

- protocol name: HTTP , DNS , FTP , SSH , TLS
- request and response messages
- commands such as GET , POST , USER , PASS
- hostnames, URLs, headers, banners
- status codes and error messages

Useful filters: http , dns , ftp , ssh , rdp , smtp , ntp

| The protocol name tells you the service. The payload tells you what happened.

Layer 7 attacks: web and remote access

Protocol	Attack	Defense
HTTP	credential theft	HTTPS, secure cookies
FTP	password capture	SFTP / FTPS
SSH	brute force	key auth, MFA, fail2ban
RDP	exposed desktop	VPN, NLA, lockout

Layer 7 attacks: infrastructure

Protocol	Attack	Defense
SMTP	phishing, open relay	SPF, DKIM, DMARC
SNTP/NTP	time spoofing	trusted time source
SNTP/NTP	amplification	restrict public access
DNS	spoofing, poisoning	DNSSEC, DoH/DoT

DEMO: HTTP is readable

1. Open Wireshark
2. Start capture
3. Visit: `http://neverssl.com`
4. Filter: `http`
5. Right-click -> Follow -> TCP Stream

Look for: `GET` , `Host` , readable headers, readable response.

Attack: plaintext traffic can be read or modified.

Defense: `HTTPS` , HSTS, secure cookies.

DEMO: FTP shows why plaintext is dangerous

Use only in a lab VM or controlled demo server.

1. Start capture
2. Connect to a test FTP server
3. Filter: ftp
4. Look for USER and PASS commands

Look for:

- username in cleartext
- password in cleartext
- commands: `LIST`, `RETR`, `STOR`

Defense: replace FTP with `SFTP` or `FTPS`.

DEMO: SSH hides the commands

1. Start capture
2. Connect to a lab host with SSH
3. Filter: `ssh or tcp.port == 22`
4. Compare it with FTP

Look for: version exchange, key exchange, encrypted payload.

Attack: brute force and stolen private keys.

Defense: key login, MFA, disable password auth, restrict source IPs.

DEMO: RDP as remote GUI traffic

1. Start capture
2. Connect to a Windows lab VM with Remote Desktop
3. Filter: `tcp.port == 3389`
4. Inspect connection setup

Look for: port `3389`, negotiation traffic, encrypted session data.

Attack: internet-exposed RDP brute force.

Defense: VPN, NLA, account lockout, no direct exposure.

DEMO: SNTP/NTP time sync

1. Start capture
2. Run: `w32tm /resync`
3. Filter: `ntp or udp.port == 123`

What students should see:

- client request
- server response
- time data used by the machine

Security reason:

- logs, certificates, Kerberos, and investigations depend on correct time

Defense:

- trusted time source, authenticated NTP where available, restrict NTP exposure

Layer 6: Presentation layer

Presentation is about how data is represented.

Examples:

- encryption: SSL , TLS
- text encoding: ASCII , UTF-8
- image formats: JPG , PNG
- document/data formats: JSON , XML , PDF
- compression: gzip , deflate

Layer 6 answers:

Can the receiver understand the format?

Is the data compressed?

Is the data encrypted?

Presentation: three jobs

Layer 6 has three major responsibilities:

```
Encoding      -> how data becomes readable/understandable
Compression   -> how data becomes smaller
Encryption    -> how data becomes unreadable to outsiders
```

Examples:

- Encoding: ASCII , UTF-8 , Base64
- Compression: gzip , deflate , Brotli
- Encryption: TLS , modern HTTPS

Presentation changes how data is represented before transport carries it.

Encoding

Encoding decides how bytes become characters or structured data.

Examples: ASCII , UTF-8 , Base64, URL encoding, JSON escaping.

What can go wrong:

- broken characters
- parser confusion
- filter bypass
- injection through unexpected encoding

Defenses:

- normalize input
- validate after decoding

DEMO: Encoding in web traffic

1. Open browser developer tools
2. Visit a page with query parameters
3. Capture traffic in Wireshark
4. Filter: http
5. Look for encoded characters in URLs or headers

Examples:

```
space      -> %20
@          -> %40
hello=one two -> hello=one%20two
Authorization: Basic <base64-data>
```

Teaching point: encoded does not mean encrypted. Base64 is reversible.

Compression

Compression reduces size before transport.

Examples: `gzip`, `deflate`, Brotli, ZIP-style compression.

Where students see it:

```
Accept-Encoding: gzip, deflate, br  
Content-Encoding: gzip
```

Benefits: faster downloads, lower bandwidth use.

Risk: side channels when secrets and attacker-controlled text are compressed together.

DEMO: Compression headers

1. Open Wireshark
2. Visit a normal HTTPS site
3. Also open browser developer tools -> Network
4. Inspect request and response headers

Look for:

```
Accept-Encoding: gzip, deflate, br  
Content-Encoding: gzip  
Content-Encoding: br
```

Wireshark note:

- with HTTPS, headers are encrypted inside TLS
- browser dev tools can show decoded application details

Encryption

Encryption protects data from being read or changed in transit.

Examples: TLS 1.2 , TLS 1.3 , HTTPS, SSH encryption, VPN tunnels.

Encryption should provide:

- confidentiality
- integrity
- authentication

What can go wrong:

- old SSL
- weak cipher suites
- expired or fake certificates

SSL and TLS

Teach this clearly:

- **SSL** is the older, insecure predecessor
- **TLS** is the modern standard
- people still say "SSL certificate", but modern HTTPS uses TLS

TLS provides:

- encryption
- integrity
- server authentication
- optional client authentication

In OSI language, TLS is strongly Presentation-layer behavior, with some Session-layer behavior.

DEMO: TLS handshake

1. Start Wireshark
2. Visit: `https://example.com`
3. Filter: `tls`
4. Inspect Client Hello and Server Hello

What students should see:

- supported TLS versions
- cipher suites
- server name indication if visible
- certificate exchange details
- encrypted application data after the handshake

Defense lesson:

Practical Cyber Security: From First Principles -- Networking Block -- Day 2

- use TLS 1.2/1.3, valid certificates, strong ciphers, HSTS

DEMO: JPG and PNG as presentation formats

1. Visit a plain HTTP page with images, if available
2. Filter: http
3. Look for Content-Type headers

Examples:

```
Content-Type: image/jpeg  
Content-Type: image/png  
Content-Encoding: gzip  
Content-Type: application/json
```

Teaching point:

- Application requested the content
- Presentation defines how the content is encoded or formatted

Layer 6 attacks and defenses

- **SSL/TLS** : downgrade, weak cipher, expired cert
- Defense: TLS 1.2/1.3, strong config, valid certs
- **JPG/PNG/PDF** : malicious parser exploit
- Defense: patch viewers, scan files, sandbox
- **JSON/XML** : parser abuse, XXE
- Defense: safe parsers, disable XXE
- Compression: side-channel leakage
- Defense: avoid mixing secrets with attacker-controlled compressed text

Layer 5: Session layer

Session is about maintaining a conversation over time.

Examples:

- web login sessions
- cookies and session IDs
- TLS session resumption
- SMB sessions
- RPC sessions
- RDP interactive sessions
- SSH login sessions

Layer 5 asks:

DEMO: Web session cookies

Use a safe test site or local lab app.

1. Open browser developer tools
2. Log in to a lab web app
3. Inspect cookies and session storage
4. Capture traffic in Wireshark

What students should connect:

- browser stores session token
- token is sent with later requests
- server uses it to recognize the same user

Attack: hijacking, fixation, stolen cookies.

Defense: `HttpOnly`, `Secure`, `SameSite`, short expiry, re-auth.

DEMO: TLS session resumption

1. Open Wireshark
2. Visit the same HTTPS site twice
3. Filter: `tls`
4. Compare the handshakes

What students may see:

- session ticket
- shorter resumed handshake
- encrypted application data after session setup

Teaching point:

- a session is not just one packet
- systems optimize repeated secure conversations

HTTP vs HTTPS in the unified model

Application service: "get me a web page"
Presentation detail: "is the content readable or encrypted?"
Session behavior: "how is this exchange maintained?"

- HTTP sends readable requests and responses
- HTTPS still serves web content, but with encryption via TLS
- Same user goal, different presentation and session properties

Warning! If students only learn "HTTP bad, HTTPS good", they miss why the models split these responsibilities differently.

DEMO: Plain HTTP capture

1. Open Wireshark
2. Start capture on your active interface
3. Visit: `http://neverssl.com`
4. Stop capture
5. Filter: `http`
6. Right-click one packet -> Follow -> TCP Stream

What students should see:

- `GET / HTTP/1.1`
- `Host: neverssl.com`
- readable headers
- readable server response

This is Application-layer content exposed on the wire.

DEMO: HTTPS and TLS capture

1. Start a fresh capture
2. Visit: `https://example.com`
3. Filter: `tls`
4. Look for Client Hello / Server Hello

What students should see:

- server name indication may still be visible
- certificates and handshake messages
- encrypted application payload after setup

Suggested filters:

- `http`
- `tls`

Protocol-to-layer recap

OSI Layer 7 Application:

HTTP, HTTPS, FTP, SSH, RDP, SMTP, SNMP/NTP, DNS, DHCP

OSI Layer 6 Presentation:

SSL, TLS, JPG, PNG, JSON, XML, compression, encoding

OSI Layer 5 Session:

cookies, session IDs, TLS session tickets, SSH/RDP sessions

TCP/IP Application:

all of the above grouped together

What OSI explains better here

- Why TLS feels like more than "just a protocol"
- Why encrypted and unencrypted web traffic are both still application behavior
- Why formatting, encoding, and session state matter even when TCP/IP hides them inside one layer

Day 2 lab checklist

Students should capture and identify at least five:

- HTTP request
- TLS handshake
- DNS query from prior lesson
- SSH or RDP connection setup
- NTP time sync
- FTP only in a controlled lab
- Content-Type such as image/jpeg , image/png , Or application/json

For each one, classify:

- OSI layer
- TCP/IP layer

Day 2 close

1. `OSI` and `TCP/IP` are two views of the same stack
2. `DHCP` gives your IP, and `DNS` gives the destination IP
3. `IP` belongs to `OSI Layer 3` / `TCP/IP Internet`
4. Layer 7 contains user-facing protocols like `HTTP` , `FTP` , `SSH` , `RDP` , `SMTP` , and `NTP`
5. Layer 6 explains encryption and formats like `TLS` , `JPG` , `PNG` , `JSON` , and compression
6. Layer 5 explains sessions, cookies, login state, and secure resumption
7. Wireshark lets us connect application traffic to the IP addresses underneath

Tomorrow: we move down one step and teach the `Transport` layer as the actual conversation underneath the app.